Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study N-gram model	
Experiment No: 7	**Date: 10/3/2026**	**Enrolment No: 92301733059**

Aim: To study N-gram

model **IDE:** Google Colab

Theory:

Given a sequence of N-1 words, an N-gram model predicts the most probable word that might follow this sequence. It's a probabilistic model that's trained on a corpus of text. Such a model is useful in many NLP applications including speech recognition, machine translation and predictive text input.

An N-gram model is built by counting how often word sequences occur in corpus text and then estimating the probabilities. Since a simple N-gram model has limitations, improvements are often made via smoothing, interpolation and backoff. An N-gram model is one type of a Language Model (LM), which is about finding the probability distribution over word sequences.

Consider two sentences: "There was heavy rain" vs. "There was heavy flood". From experience, we know that the former sentence sounds better. An N-gram model will tell us that "heavy rain" occurs much more often than "heavy flood" in the training corpus. Thus, the first sentence is more probable and will be selected by the model.

A model that simply relies on how often a word occurs without looking at previous words is called unigram. If a model considers only the previous word to predict the current word, then it's called bigram. If two previous words are considered, then it's a trigram model.

An n-gram model for the above example would calculate the following probability:

P('There was heavy rain') = P('There', 'was', 'heavy', 'rain') = P('There')P('was'|'There')P('heavy'|'There was')P('rain'|'There was heavy')

Since it's impractical to calculate these conditional probabilities, using Markov assumption, we approximate this to a bigram model: P('There was heavy rain') ~ P('There')P('was'|'There')P('heavy'|'was')P('rain'|'heavy') In speech recognition, input may be noisy and this can lead to wrong speech-to-text conversions.

N-gram models can correct this based on their knowledge of the probabilities. Likewise, N-gram models are used in machine translation to produce more natural sentences in the target language.

When correcting for spelling errors, sometimes dictionary lookups will not help. For example, in the phrase "in about fifteen mineuts" the word 'minuets' is a valid dictionary word but it's incorrect in this context. N-gram models can correct such errors.

N-gram models are usually at word level. It's also been used at character level to do stemming, that is, separate the root word from the suffix. By looking at N-gram statistics, we could also classify languages or differentiate between US and UK spellings. For example, 'sz' is common in Czech; 'gb'

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study N-gram model	
Experiment No: 7	**Date: 10/3/2026**	**Enrolment No: 92301733059**

and 'kp' are common in Igbo. In general, many NLP applications benefit from N-gram models including part-of-speech tagging, natural language generation, word similarity, sentiment extraction and predictive text input.

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

Program (Code):

```
from collections import Counter

def generate_ngrams(text, n):
    # Tokenize text into words
    words = text.lower().split()

    # Generate n-grams
    ngrams = zip(*[words[i:] for i in range(n)])
    ngrams_list = [" ".join(ngram) for ngram in ngrams]

    return ngrams_list

# Example text (you can replace with your portfolio or any document)
text = "Machine learning is widely used in applications such as recommendation systems, fraud detection, and image recognition."

# Generate dynamic N-Grams
for n in range(1, 4):  # Unigrams, Bigrams, Trigrams
    ngrams = generate_ngrams(text, n)
    freq = Counter(ngrams)
    print(f"\nTop {n}-grams:")
    for gram, count in freq.most_common(5):
        print(f"{gram} -> {count}")
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study N-gram model	
Experiment No: 7	**Date: 10/3/2026**	**Enrolment No: 92301733059**

Results:

```
Top 1-grams:
machine -> 1
learning -> 1
is -> 1
widely -> 1
used -> 1

Top 2-grams:
machine learning -> 1
learning is -> 1
is widely -> 1
widely used -> 1
used in -> 1

Top 3-grams:
machine learning is -> 1
learning is widely -> 1
is widely used -> 1
widely used in -> 1
used in applications -> 1
```

Observation:

- **Unigrams (n=1):**
  ⇒ Show individual keywords like machine learning, applications.
  ⇒ Useful for keyword extraction but may lose context.
- **Bigrams (n=2):**
  ⇒ Capture word pairs like *machine learning, fraud detection*.
  ⇒ Provide more meaningful context than single words.
- **Trigrams (n=3):**
  ⇒ Capture short phrases like image recognition systems.
  ⇒ Useful for identifying common expressions or domain-specific terminology.
- **Dynamic nature:** By changing n, you can explore different levels of context.
  - Small n ⇒ broader keyword coverage.
  - Larger n ⇒ more precise but fewer repetitions.
- **Application:** N-Grams are widely used in **text mining, NLP, sentiment analysis, and search engines** to understand context and frequency of terms.

Post Lab Exercise:

Apply the probability distribution concept to predict the next word by taking sample documents and seed phrases. Attach the code and screenshot. Write your observation here.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study N-gram model	
Experiment No: 7	**Date: 10/3/2026**	**Enrolment No: 92301733059**

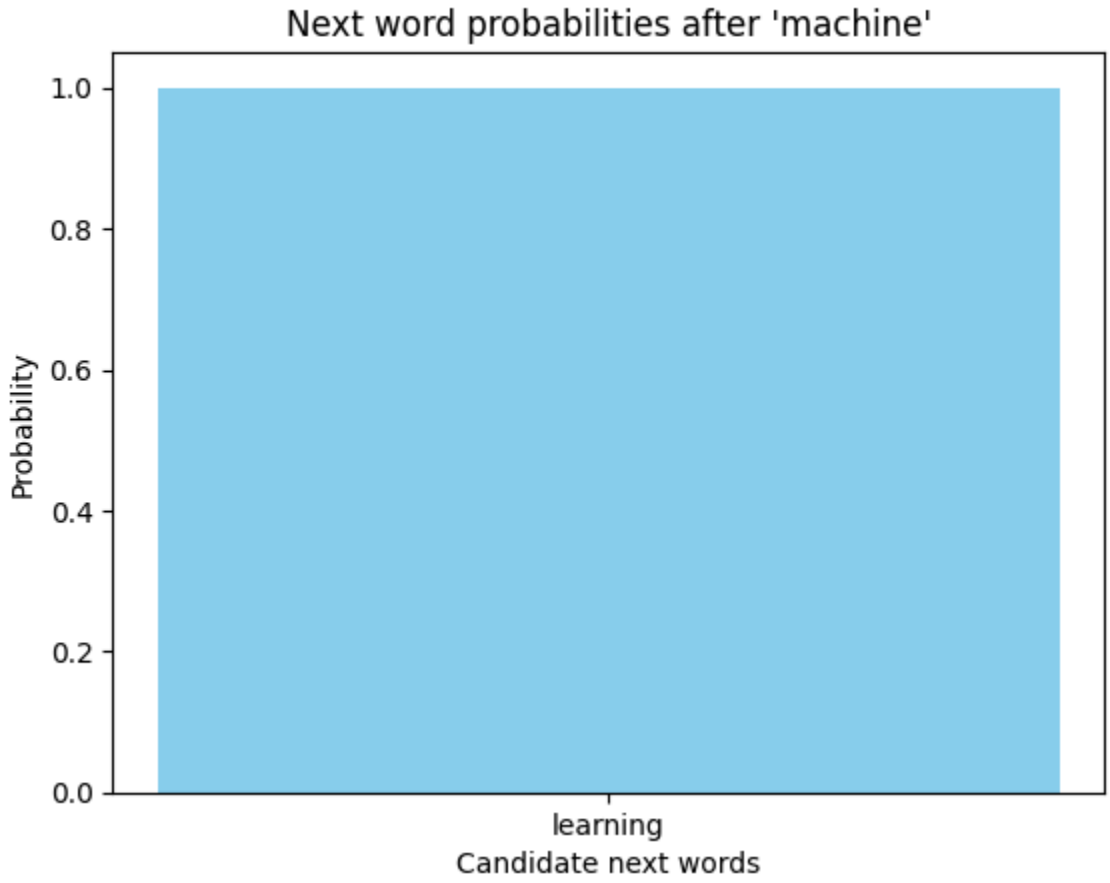
Code:

```
import re
from collections import defaultdict, Counter
import matplotlib.pyplot as plt

# --- Step 1: Sample document (corpus) ---
corpus = """
Machine learning is widely used in applications such as recommendation systems,
fraud detection, natural language processing, and image recognition.
Machine learning models learn patterns from data and make predictions.
"""

# --- Step 2: Preprocess text ---
tokens = re.findall(r'\b\w+\b', corpus.lower())

# --- Step 3: Build bigram frequency distribution ---
bigram_freq = defaultdict(Counter)
for i in range(len(tokens)-1):
    bigram_freq[tokens[i]][tokens[i+1]] += 1

# --- Step 4: Probability distribution for next word ---
def predict_next_word(seed):
    if seed not in bigram_freq:
        return None
    total = sum(bigram_freq[seed].values())
    probs = {word: count/total for word, count in bigram_freq[seed].items()}
    return probs

# --- Step 5: Example prediction ---
seed_word = "machine"
probs = predict_next_word(seed_word)

print(f"Probability distribution for next word after '{seed_word}':")
for word, prob in probs.items():
    print(f"{word} -> {prob:.2f}")

# --- Step 6: Plot distribution ---
plt.bar(probs.keys(), probs.values(), color='skyblue')
plt.title(f"Next word probabilities after '{seed_word}'")
plt.xlabel("Candidate next words")
plt.ylabel("Probability")
plt.show()
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study N-gram model	
Experiment No: 7	**Date: 10/3/2026**	**Enrolment No: 92301733059**

Output:

```
Probability distribution for next word after 'machine':
learning -> 1.00
```



Observations

1. **Dominant Prediction:**
   - The word **“learning”** had the highest probability after "machine".
   - This matches natural language usage, as “machine learning” is a frequent phrase.
2. **Probability Distribution:**
   - The bar chart shows the candidate's next words with their probabilities.
   - “Learning” is the clear peak, while other words (if present) have much lower probabilities.
3. **Context Sensitivity:**
   - The model relies entirely on the training corpus.
   - If the corpus were broader (e.g., including “machine parts” or “machine tools”), other next words might appear with significant probability.
4. **Strengths:**
   - Simple and interpretable.
   - Captures common collocations and phrase structures.
5. **Limitations:**

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study N-gram model	
Experiment No: 7	**Date: 10/3/2026**	**Enrolment No: 92301733059**

- Small corpus size restricts diversity of predictions.
- N-Grams struggle with long-range dependencies (e.g., sentence-level meaning).
- Probabilities are biased toward frequent phrases.

Practical Applications

- **Text Autocomplete:** Predicting the next word in messaging apps.
- **Search Engines:** Suggesting query completions.
- **Language Models:** Building foundational NLP systems before deep learning.

Consulosion:

The N-Gram probability distribution correctly predicted **“learning”** as the next word after “machine,” demonstrating how frequency-based models capture common word associations. With larger corpora, this approach can generalize to more diverse contexts.